KSRCE – GKT

AI Supercomputing Lab-as-a-Service

Technical Implementation & SDLC Document

Document Version	1.0 — Beta
Date	March 08, 2026
Classification	Confidential — Internal Use Only
Prepared By	Punith — Engineer, Global Knowledge Technologies
Stakeholders	GKT Engineering, KSRCE Administration

Executive Summary

This document serves as the unified technical blueprint for the KSRCE–GKT AI Supercomputing Lab-as-a-Service (LaaS) platform — a first-of-its-kind on-premises solution that delivers fractional GPU desktop computing, ephemeral compute sessions, and full-machine exclusive access to a fleet of 4 high-performance nodes, each powered by AMD Ryzen 9 9950X3D CPUs and NVIDIA RTX 5090 32GB GPUs. The platform targets university students, faculty, researchers, and public users seeking on-demand GPU compute delivered through a browser-based interface.

The document is structured in two parts:
Part I covers the complete infrastructure architecture — hardware, GPU virtualisation strategy, storage, networking, and phased burn-in protocols.
Part II covers the software development lifecycle (SDLC) including the sprint plan, module breakdown, technology stack, integration dependencies, and delivery timeline aligned with the March 13, 2026, development completion target and the March 15–17 deployment window.

SPRINT TIMELINE — ACCELERATED DELIVERY

Development Sprint: March 08 – March 13, 2026 (6 days)
Final QA & Load Testing: March 15, 2026
Soft Launch (Closed Beta — KSRCE faculty + select students): March 16, 2026
Public Launch & Marketing Push: March 17, 2026

All core deliverables must be production-ready by EOD March 13. Quality is non-negotiable.

Fleet Capacity at a Glance

Resource	Per Node	Fleet Total (4 Nodes)
CPU Cores	16C / 32T (Ryzen 9 9950X3D)	64C / 128T
System RAM	64 GB DDR5-6000	256 GB
GPU	RTX 5090 — 32 GB VRAM	128 GB VRAM
NVMe Storage	2 TB (7,250 MB/s)	8 TB
Max Concurrent GPU Desktop Sessions	4–8 (fractional)	16–32
Max Concurrent Ephemeral Sessions	6–10	24–40

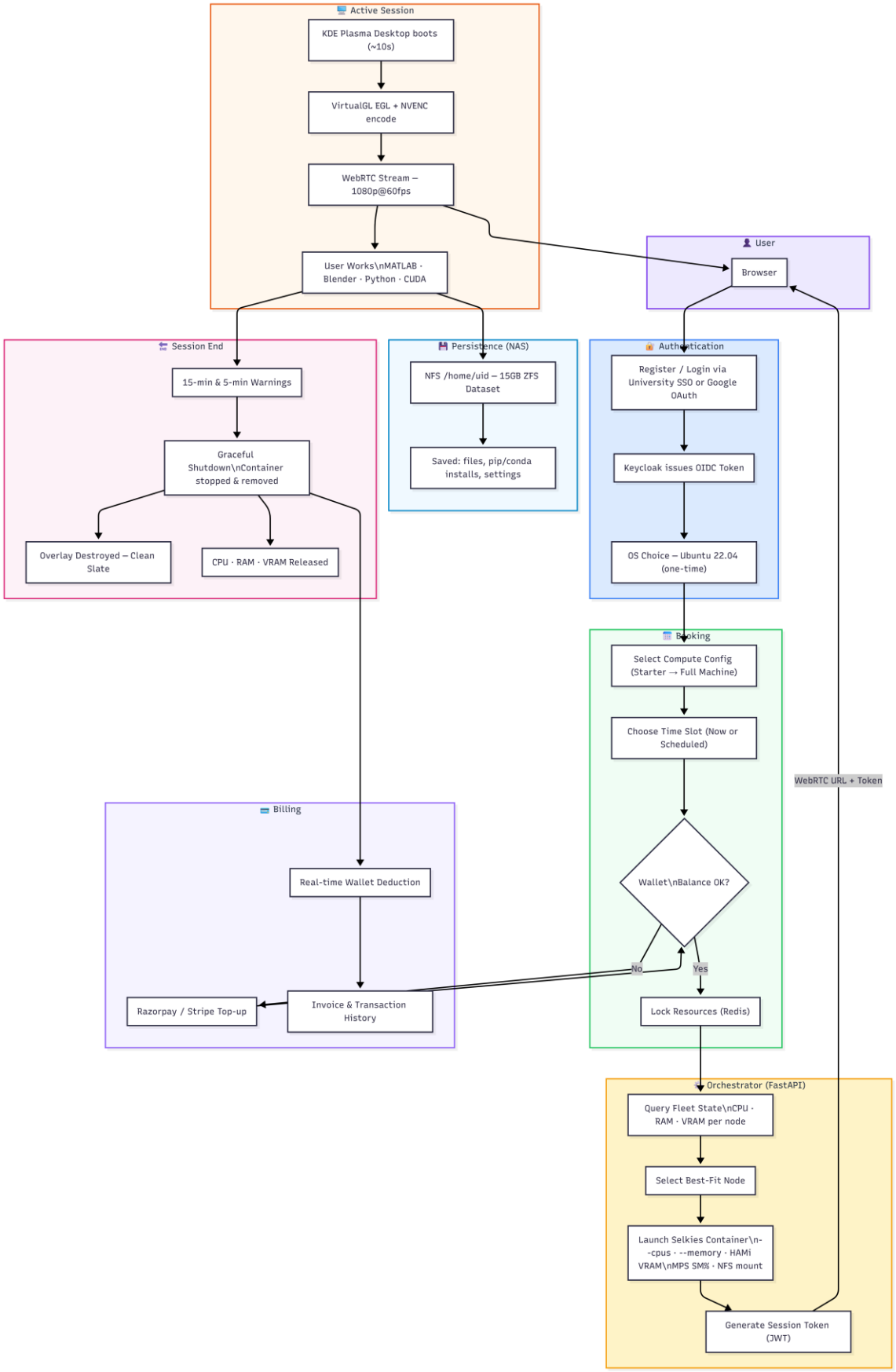

Figure: **LaaS Session Lifecycle: End-to-End Data Flow**

PART I — Infrastructure Architecture & Setup

1. Hardware Inventory

1.1 Compute Nodes (4 Identical Machines)

Component	Specification
CPU	AMD Ryzen 9 9950X3D (16C/32T, 3D V-Cache, 144MB L3)
Motherboard	ASUS ProArt X670E-Creator WiFi DDR5
GPU	Zotac RTX 5090 Solid OC 32GB GDDR7 (Blackwell, 170 SMs)
RAM	G.SKILL DDR5 Trident Z5 NEO 6000MHz 64GB (2×32GB) CL30
Storage	Samsung 990 EVO Plus NVMe 2TB (7,250 MB/s sequential)
PSU	Corsair AX1600i 1600W (80+ Titanium)
Cooling	Corsair Nautilus RS ARGB 360mm AIO
Cabinet	Corsair 3500X (3 × ARGB fans)

1.2 Additional Procurement Required

PROCUREMENT — Items Required Before Phase 0

The following components are NOT yet available and must be procured before infrastructure deployment can begin. Estimated total: ₹2,25,000 – ₹3,50,000.

Category	Item	Qty	Est. Cost (₹)
Networking	Intel X550-T1 10GbE PCIe NIC	4	26,000
Networking	Mikrotik CRS309-1G-8S+ 10GbE SFP+ Switch	1	25,000
Networking	SFP+ DAC Cables	6	5,000
Networking	TP-Link TL-SG108E 8-port 1GbE Managed Switch	1	3,000
GPU/Display	HDMI 2.1 4K Dummy Dongles	4	2,800
NAS (5th Machine)	ATX System + 4×4TB WD Red Plus + 10GbE NIC + 16GB RAM	1	~80,000
(OR Alternative)	Synology DS923+ with 10GbE Expansion	1	~1,20,000
Power	5kVA Online UPS	1	40,000–80,000
Environment	Server Room AC / Mini-Split (3–4kW heat)	1	Varies
Security	Physical Lock / Access Control for Server Room	1	Varies

2. The Consumer GPU Challenge — RTX 5090

The RTX 5090 is a GeForce consumer GPU. This is a foundational architectural constraint that shaped every infrastructure decision. Understanding these limitations is critical to appreciating the chosen architecture.

CRITICAL — Consumer GPU Limitations (RTX 5090 / GeForce)

- No NVIDIA vGPU — requires RTX PRO 6000 / L40S / A100 (enterprise licensing)
- No MIG (Multi-Instance GPU) — only available on A100 / H100 / B200
- No SR-IOV GPU Partitioning — not supported on GeForce cards
- No Hyper-V GPU-P — Windows Server 2025 explicitly excludes consumer GPUs
- vgpu_unlock hack does NOT work on Blackwell architecture

Impact: GPU cannot be hardware-partitioned across VMs. All GPU sharing must be software-based (containers + CUDA MPS + HAMi-core) or all-or-nothing VM passthrough.

2.1 The D3cold Reset Bug (Confirmed)

After a GPU-passthrough VM shuts down, the RTX 5090 can enter a D3cold power state and fail to wake. The host CPU experiences a soft lockup, and only a full machine reboot recovers the GPU. This bug has been confirmed by NVIDIA and is documented across Proxmox forums, Tom's Hardware, and igor'sLAB. A $1,000 community bounty was posted for a fix.

Impact: VM-based GPU passthrough (vfio-pci) is unreliable for production multi-tenant sequential sessions without extensive mitigation. This finding directly drove the architectural pivot from VM-centric to container-centric GPU delivery.

2.2 Architectural Pivot — From VMs to Unified Containers

ARCHITECTURAL DECISION — The Pivot

Initial Design (v1): KVM VMs for stateful desktops (CPU-only, GPU passthrough for Tier 1). Ephemeral containers for Jupyter/CLI.

Problem: Consumer GPU cannot be split across VMs. Only 1 GPU user per VM passthrough. D3cold reset bug makes sequential GPU passthrough unreliable.

Final Design (v2 — Current): ALL user sessions (CPU-only and GPU-accelerated, stateful and ephemeral) run as Selkies EGL Desktop containers with software-enforced GPU fractioning via HAMi-core + CUDA MPS. No KVM VMs for user compute. Proxmox is retained for host node management only.

Result: 4–8 concurrent GPU desktop users per node vs. 1 under the VM model. Full fractional VRAM pricing (4GB / 8GB / 16GB / 32GB) becomes viable.

3. Chosen Architecture — Unified Selkies Container Model

All 4 nodes are identically configured. NVIDIA drivers are loaded on the host at all times. No vfio-pci mode. The D3cold bug is completely avoided because the GPU never changes ownership — it is always driven by the host. Three breakthrough technologies combine to deliver fractional GPU desktops on consumer hardware:

3.1 The Three Enabling Technologies

Technology	Role	Key Capability
Selkies EGL Desktop	GPU-Shared Desktop Containers	Full KDE Plasma desktop inside Docker. VirtualGL EGL backend for GPU-accelerated rendering. WebRTC streaming to browser at 60fps. Explicitly supports sharing one GPU across many containers.
HAMi-core (libvgpu.so)	CUDA API Interception	Intercepts cuMemAlloc to enforce hard VRAM limits per container. nvidia-smi inside container reports limited VRAM (e.g., 4GB instead of 32GB). No kernel module required. Works on consumer GPUs.
CUDA MPS	Compute Partitioning	Per-client VRAM caps (second enforcement layer). SM partitioning — RTX 5090 has 170 SMs allowing fine-grained compute allocation. Isolated GPU virtual address spaces per client (Volta+). Supports up to 48 concurrent CUDA contexts.

WHY DUAL-LAYER VRAM ENFORCEMENT

HAMi-core (Layer 1) catches allocation attempts at the CUDA API level and makes nvidia-smi report the correct limited VRAM. CUDA MPS (Layer 2) enforces hard caps at the driver level — even if HAMi-core is bypassed by a statically-linked binary, MPS still enforces the limit. Together, they provide defense-in-depth that neither alone can achieve.

3.2 Compute Configuration Tiers (example pricing)

Stateful GUI Desktop Configs (All delivered as Selkies EGL Desktop containers):

Config	vCPU	RAM	GPU VRAM	Best For	Price/hr
Starter	2	4 GB	None	Light coding, docs	₹15
Standard	4	8 GB	None	MATLAB, development	₹30
Pro	4	8 GB	4 GB	Blender, ML inference	₹60
Power	8	16 GB	8 GB	Heavy sim, 3D render	₹100
Max	8	16 GB	16 GB	Large model training	₹150
Full Machine	16	48 GB	32 GB (exclusive)	DL research, CUDA dev	₹300

Ephemeral Compute Configs (Docker Containers — Jupyter / Code-Server / SSH):

Config	vCPU	RAM	GPU VRAM	Best For	Price/hr
Ephemeral CPU	2	4 GB	None	Jupyter, Code-Server, SSH	₹10
Ephemeral GPU-S	2	4 GB	4 GB	Light ML inference	₹40
Ephemeral GPU-M	4	8 GB	8 GB	Standard ML training	₹75
Ephemeral GPU-L	8	16 GB	16 GB	Heavy ML, large models	₹120

3.3 Isolation Model

Isolation Layer	Level	Mechanism
CPU / RAM	Hard (kernel-enforced)	cgroups v2 (--cpus / --memory). Cannot be exceeded.
Filesystem	Hard	Docker read-only image layers + overlay. User /home on NFS.
Process	Strong	PID namespaces. Users cannot see each other's processes.
Network	Strong	Network namespaces + VLAN segmentation.
GPU VRAM	Enforced (software)	HAMi-core + MPS dual-layer. CUDA_ERROR_OUT_OF_MEMORY on exceed.
GPU Compute (SM)	Enforced (software)	HAMi-core + MPS ACTIVE_THREAD_PERCENTAGE.
GPU Address Space	Isolated	MPS Volta+ per-client isolated GPU virtual address spaces.
GPU L2 / Bandwidth	Shared	Cannot partition on consumer GPUs. 10–30% variability possible.
GPU Fatal Fault	Propagates	MPS auto-recovers. Watchdog restarts containers in 30–60s.
Base Software	Absolute	Docker read-only layers. No root/sudo. Immutable.

CONSIDERATION — GPU Noisy Neighbour & Fault Propagation

Users on shared GPU tiers (Pro/Power/Max) may experience 10–30% performance variability under peak co-tenancy due to shared L2 cache and memory bandwidth. A GPU fatal fault from one user's CUDA operation will affect co-resident containers — MPS auto-recovers in 30–60s with no data loss (user files are on NAS). Full Machine tier is immune to both effects.

Recommended ToS Language: "GPU Desktop sessions share a physical GPU for cost efficiency. In rare circumstances (<0.1%), a GPU interruption may require a 30–60 second automatic

restart. Your files are never affected. For guaranteed uninterrupted GPU access, use Full Machine."

4. Storage Architecture

4.1 Per-Node NVMe Partitioning (2TB)

Partition	Size	Filesystem	Purpose
nvme0n1p1	128 GB	EXT4	Proxmox OS + swap (8 GB)
nvme0n1p2	500 GB	EXT4	/var/lib/docker — images, container overlays
nvme0n1p3	1,100 GB	LVM PV	VG_CONTAINERS — ephemeral scratch, wiped on session end
nvme0n1p4	240 GB	EXT4	/shared — read-only datasets, software caches

HARD DATA ISOLATION

Docker storage (p2) and VG_CONTAINERS (p3) are on separate NVMe partitions. Even if VG_CONTAINERS is completely filled or corrupted, Docker storage is untouched. User persistent data lives on the NAS via NFS — completely independent of both local partitions. Ephemeral session corruption can never reach user data or base images.

4.2 Centralised NAS (5th Machine)

- TrueNAS Scale (Debian-based, ZFS native) — recommended over Synology for flexibility
- ZFS Pool: 4×4TB HDDs in RAIDZ1 (~12TB usable)
- Per-user ZFS dataset with 15GB hard quota: datapool/users/<uid>
- NFS export over 10GbE to all 4 compute nodes (nconnect=4 for ~3–4 Gbps throughput)
- Automated ZFS snapshots every 6 hours (7 daily + 4 weekly retained)
- Users can self-recover files via .zfs/snapshot/ directory

5. Networking Architecture

VLAN	Subnet	Purpose	Interface
10 (Management)	10.10.10.0/24	Proxmox Web UI, SSH, cluster comms	Onboard 2.5GbE
20 (Stateful)	10.10.20.0/24	Stateful desktop container traffic, WebRTC	10GbE
30 (Ephemeral)	10.10.30.0/24	Jupyter, Code-Server, SSH sessions	10GbE
40 (Storage)	10.10.40.0/24	NFS traffic (NAS ↔ Nodes). No internet.	10GbE
50 (Services)	10.10.50.0/24	Portal, Keycloak, Reverse Proxy	10GbE

- External Access: Cloudflare Tunnel (browser, zero open ports) + Tailscale (SSH, Moonlight)
- Internet: Minimum 500 Mbps symmetric upstream. Dual ISP recommended for redundancy.
- TURN Server: coturn deployed for WebRTC NAT traversal through university firewalls.

BLOCKER — ISP Verification Required

Verify that the ISP at the deployment location can provide ≥500 Mbps symmetric upstream. Obtain quotes from 2 ISPs. Budget ₹30,000–80,000/month for business-grade symmetric fibre. Single ISP failure = platform offline for all remote users.

6. Phase 0 — Burn-In Protocol

Phase 0 is the decision gate that validates the chosen architecture before fleet-wide deployment. All testing is performed on a single node first.

Week	Focus Area	Key Activities
	Hardware Validation	BIOS config, memtest86+ (48h), Proxmox install, cluster formation, 10GbE + VLAN setup, NAS ZFS + NFS configuration, NFS I/O benchmark (fio)
Week 1	Architecture A Burn-in	NVIDIA driver on host, Docker + nvidia-container-toolkit, Selkies EGL containers, HAMi-core + MPS integration, multi-container GPU sharing, NVENC concurrent stream testing (4/6/8 sessions), nvidia-patch for NVENC limits
	Stress & Decision Gate	48-hour stress test (4 containers with fractional VRAM), fault injection (deliberate GPU crash → verify MPS recovery 10×), MPS watchdog service, DECISION: confirm Architecture A or fall back to B

DECISION GATE CRITERIA

Architecture A is confirmed production-ready if:
✓All 4 fractional VRAM combinations work (4+8+8+8, 4+4+8+16, 2×16, etc.)
✓48-hour stress test passes without crashes or VRAM limit violations
✓MPS fault recovery succeeds in 10/10 fault injection tests
✓NVENC supports ≥4 concurrent desktop streams
✓CPU-only containers stream smoothly via WebRTC

If any test fails → investigate root cause → attempt fix → if unresolvable, fall back to Architecture B (VM passthrough, one GPU per VM, no fractional tiers).

7. Infrastructure Risks & Considerations

Risk	Severity	Mitigation
RTX 5090 D3cold Reset Bug	HIGH	Eliminated by Architecture A (GPU never changes ownership). Relevant only if Architecture B fallback is used.
GPU Noisy Neighbour	MEDIUM	10–30% variability disclosed in ToS. Full Machine tier offered for guaranteed performance.
GPU Fatal Fault Propagation	MEDIUM	MPS auto-recovery + watchdog service + admin alerts. User data safe on NAS.
Ryzen 9950X3D + Proxmox Stability	MEDIUM	Disable XMP during burn-in. Update BIOS + AMD microcode. Pin to stable kernel 6.8.x LTS.
NVENC Session Limit (Consumer)	MEDIUM	nvidia-patch removes firmware limit. Must verify on RTX 5090 in Phase 0.
OEM Software Licensing(Adobe, MATLAB…etc)	HIGH (Blocker)	Must resolve before building base images. Per-seat license is NOT valid for shared hosting.
ISP Bandwidth Insufficiency	HIGH (Blocker)	Verify ≥500 Mbps symmetric. Obtain quotes from 2 ISPs before launch.
Thermal Management (24/7 Operation)	MEDIUM	DCGM exporter monitors temp. Alert at 80°C, emergency at 90°C. Consider Noctua industrial fans.
Crypto Mining Abuse	LOW–MEDIUM	GPU utilisation pattern monitoring. Block mining pool IPs. Kill sustained >95% GPU with no CUDA context.
Single NVMe Failure	HIGH	User data on NAS (ZFS snapshots). Local NVMe is ephemeral/rebuild-able. No single point of data loss.

PART II — Software Development Lifecycle & Sprint Plan

8. Sprint Timeline Overview

ACCELERATED TIMELINE — Speed to Market

The AI infrastructure market is consolidating rapidly. TCS + OpenAI + AMD are entering the Indian data centre space. First-mover advantage is critical. The following timeline reflects the business reality: launch by mid-March 2026 or risk strategic irrelevance.

Development Sprint: Today → Friday, March 13, 2026
QA & Load Testing: March 15
Soft Launch (Closed Beta): March 16
Public Launch: March 17

Day	Date	Sprint Focus	Deliverables
Day 1	Mar 08	Foundation & Auth	Project scaffolding, DB schema, Keycloak/Auth integration, User registration + email verification
Day 2	Mar 09	Core Platform	Role-based access control, Profile management, Dashboard layout, Wallet system backend
Day 3	Mar 10	Booking Engine	GPU slot booking workflow, Real-time availability dashboard, Resource allocation logic, Queue management
Day 4	Mar 11	Payments & Mentorship	Stripe/Razorpay integration, Subscription tiers (Bronze/Silver/Gold), Mentor booking system, Calendar scheduling
Day 5	Mar 12	Admin & Analytics	Admin dashboard, User management panel, Usage analytics, Hardware monitoring views, Invoice generation
Day 6	Mar 13	Polish & Chatbot	AI Chatbot (FAQ MVP), Responsive UI polish, Dark mode, Performance optimisation (<2s load), Integration testing

9. Technology Stack

Layer	Technology	Rationale
Frontend	Next.js (React) + TypeScript	SSR for SEO, app router, mobile-responsive. Futuristic AI-themed design with dark/light mode.
Backend API	Python FastAPI	Async-native, auto-generated OpenAPI docs, <500ms p95 response time.
State Store	Redis	Session state, real-time fleet status,

		booking lock management, pub/sub for live updates.
Primary Database	PostgreSQL	Users, bookings, billing records, audit logs. Optimised for read-heavy dashboard queries.
Authentication	Keycloak (OIDC/SAML)	SSO with university Google Workspace/LDAP. MFA enforced. Role-based groups.
Payment Gateway	Razorpay (primary) + Stripe (fallback)	Indian rupee support, UPI, auto-debit. Stripe for international/corporate users.
Container Orchestration	Docker API (via FastAPI orchestrator)	Direct Docker API calls for session lifecycle. No Kubernetes overhead for 4-node scale.
Monitoring	Prometheus + Grafana + DCGM Exporter	Node metrics, GPU health, per-user usage, billing analytics.
Remote Desktop Streaming	Selkies-GStreamer (WebRTC)	Browser-native, no client install. NVENC H.264 encoding. 60fps at 1080p.
Reverse Proxy	Traefik / Caddy	TLS termination, WebSocket passthrough, path-based routing.
Remote Access	Cloudflare Tunnel + Tailscale	Zero open ports. Works through university firewalls.

10. Platform Module Breakdown

10.1 User Authentication & Onboarding

- Signup with email + OTP verification (university email for members, Google OAuth for public)
- Keycloak-federated SSO (university Google Workspace or LDAP)
- Role assignment: Student, Faculty, Admin, Corporate Partner, Public User
- 2FA enforced via TOTP (Google Authenticator)
- Profile setup: name, department, college, semester, preferences
- Freemium tier auto-assignment for KSRCE students

10.2 GPU Booking & Resource Management

- Real-time availability dashboard showing fleet resource status per node
- Pre-defined compute config selection (Starter through Full Machine)
- Time-slot booking with 1-hour blocks (now or scheduled)
- Auto-locking of GPU nodes upon confirmed booking
- Queue management for high-demand periods
- Session launch → container spin-up → WebRTC URL delivery (~15–20s)
- Session reconnection if browser closes mid-session
- Graceful shutdown with 15-min and 5-min warnings

- Over-usage handling: prompt to extend or auto-terminate

10.3 Wallet & Payment System

- Wallet top-up via Razorpay (UPI, cards, net banking)
- Real-time wallet deduction during active sessions
- Subscription tiers: Bronze (₹500/20h), Silver (₹1,500/60h), Gold (₹3,000/150h) **(example)**
- Academic Integration Fees: per-section per-semester billing to college
- Automated low-balance notifications (web push, email, SMS)
- Recharge packages with bonus minutes
- Invoice generation and transaction history

10.4 Mentor Booking System

- Mentor profile creation (expertise, experience, ratings)
- Availability calendar with configurable time slots
- Session booking with automatic calendar scheduling
- Integrated video call (Jitsi Meet or similar)
- Booking confirmation + reminder notifications
- Automated payment and invoicing for mentor sessions

10.5 Admin Dashboard & Analytics

- Real-time fleet overview: CPU/RAM/GPU utilisation per node
- User management: CRUD, role assignment, access revocation
- Session monitoring: active sessions, duration, resource consumption
- Revenue analytics: total, per-tier, per-college, per-time-period
- GPU health: temperature, VRAM, fault events, uptime
- Audit log: all user actions, login events, session lifecycle events
- Hardware status: NVMe health (SMART), NAS capacity, network throughput

10.6 AI Chatbot (MVP)

- FAQ-driven chatbot for common queries (pricing, booking, troubleshooting)
- Escalation workflow to human support
- Deployed on the web interface (floating widget)
- Phase 2: context-aware, integrated with booking and billing systems

10.7 Responsive Web UI

- Mobile-first, dark-mode-first aesthetic (futuristic AI theme)
- Fast load times (<2s target). Lighthouse score ≥90.
- Neon accents, soft gradients, animated circuit board patterns
- Dashboard-driven layout with data visualisation
- Dark and light mode toggle

11. V1 Scope Exclusions (Phase 2 Features)

EXPLICITLY OUT OF SCOPE FOR WEEK 1 SPRINT

- Collaborative coding environments (JupyterHub multi-user)
- Mobile app (iOS / Android)
- Data annotation services platform
- AI model marketplace
- Predictive maintenance system
- Advanced gamification features
- Learning path recommendation engine
- Project showcase portal
- Industry projects marketplace

These will be phased in post-launch as Version 2.0 features.

12. Third-Party Dependencies & Integrations

Integration	Purpose	Dependency Type	Risk
Keycloak	SSO / Identity Provider	Self-hosted (Docker)	Low — mature, well-documented
Razorpay	Payment processing (INR)	External API	Low — widely used in India
Stripe	Payment (international / fallback)	External API	Low
Cloudflare	Tunnel + DDoS protection	External service	Low — free tier sufficient
Tailscale	VPN for SSH / Moonlight users	External + self-hosted	Low
Selkies-GStreamer	WebRTC desktop streaming	Open-source (Docker)	Medium — niche project
HAMi-core	GPU VRAM enforcement	Open-source (build from source)	Medium — requires Phase 0 validation
CUDA MPS	GPU compute partitioning	NVIDIA (built into CUDA)	Low — official NVIDIA component
Prometheus + Grafana	Monitoring & dashboards	Self-hosted	Low — industry standard
coturn	WebRTC TURN relay	Self-hosted	Low — handles NAT traversal
MATLAB (Network License)	Proprietary software in base image	Client/University procured	HIGH — licensing blocker

13. Development Standards

Aspect	Standard
Security	All user data encrypted at rest (LUKS on NAS) and in transit (TLS 1.3). No hardcoded credentials. JWT-based APIs. RBAC enforced.
Performance	API response times <500ms (p95). Database optimised for read-heavy dashboards. Frontend <2s load time (Lighthouse ≥90).
Availability	99% uptime target. Automated backups (ZFS snapshots). Disaster recovery plan.
Testing	Critical paths (booking, payment, auth) have automated tests. Manual QA for UI/UX.
Documentation	API documentation (auto-generated via FastAPI). Deployment runbook. Onboarding guide.
Data Protection	DPDPA 2023 compliance. Consent, data minimisation, breach notification procedures.

14. Deployment & Launch Plan

Date	Milestone	Details
Mar 08–13	Development Sprint	All 7 core modules built, tested, integration-verified.
Mar 15	QA & Load Testing	Automated test suite, load simulation (100 concurrent users), security audit (OWASP top 10 checks).
Mar 16	Soft Launch (Closed Beta)	KSRCE faculty + 20–30 select students. Monitor booking flow, payment processing, session stability.
Mar 17	Public Launch	Marketing push, open registration. Gradual ramp-up to full capacity.
Mar 18–31	Stabilisation	Bug fixes from beta feedback. Performance tuning. Monitoring alert refinement.
Apr 01+	Phase 2 Features	V2 roadmap: gamification, marketplace, collaborative coding, mobile app.

15. Full Infrastructure Timeline

The infrastructure and software timelines run in parallel. Infrastructure Phases 0–1 must complete before the platform can be fully operational, but software development can proceed concurrently using mock/simulated backends.

Phase	Duration	Focus
Phase 0: Burn-in	Mar 08–09	Hardware assembly, Proxmox install, GPU burn-in (both architectures), NAS setup, networking, DECISION GATE

Phase 1: Core Infra	Mar 10–11	Apply architecture to all nodes, build Selkies Docker image, NFS user storage automation, Keycloak deployment
Phase 2: Orchestration	Mar 11–12	FastAPI orchestrator, Selkies WebRTC integration, Cloudflare Tunnel, booking system, container lifecycle
Phase 3: Portal	Mar 10–12	Next.js frontend, Keycloak auth integration, admin panel, user registration flow
Phase 4: Billing & Monitoring	Mar 12–13	Razorpay billing, Prometheus + Grafana + DCGM, audit logging, session warnings, anti-abuse
Phase 5: Beta	Mar 15–16	10–20 beta users, load testing, security audit, documentation
Phase 6: Launch	Mar 17+	Production launch, gradual university rollout, public user access

CONSIDERATION — Timeline Alignment

The infrastructure setup and software development sprint run in parallel from March 08–13. Infrastructure burn-in (Phase 0–1) and platform development (Phases 2–4) are executed concurrently, with all core deliverables production-ready by EOD March 13.

Phase 5 (QA + closed beta) begins March 15–16 with KSRCE faculty and select students. Public launch is March 17.

Critical path: authentication, booking, payment, and at least one validated compute tier must be fully operational by launch day. Post-launch stabilisation and V2 feature rollout continue from March 18 onward.

Appendix

A. Revenue Model Summary

Revenue Stream	Description	Pricing Model
Academic Integration Fees	Per-section per-semester LaaS fee bundling GPU-hours, content, support	Pre-paid by college
Premium Student Passes	Bronze / Silver / Gold tiered packages	YTD
Pay-as-you-go Wallet	Real-time GPU consumption at standard rate	YTD
Partner College Subscriptions	Annual subscription for external colleges	Custom per-college
Mentor Booking Fees	One-on-one guidance sessions	Tiered by mentor experience

B. Security Checklist

- OTP + 2FA for all user logins
- JWT-based API authentication with short-lived tokens
- RBAC enforced at API and UI level
- HTTPS everywhere (TLS 1.3)
- Data encrypted at rest (LUKS on NAS, ZFS encryption)
- DPDPA 2023 compliance (consent, data minimisation, breach notification)
- Session idle timeout: 30min warning → 45min suspend → 60min terminate
- Container security: no-new-privileges, read-only rootfs, pids-limit, seccomp, AppArmor
- Anti-abuse: mining detection, egress rate limiting, mining pool IP blocking
- Fail2Ban on SSH, Keycloak brute-force protection

C. Glossary

Term	Definition
LaaS	Lab-as-a-Service — delivering lab compute infrastructure as a remotely accessible, on-demand service
Selkies EGL	Open-source project for GPU-accelerated Linux desktops inside Docker containers, streamed via WebRTC
HAMi-core	Open-source CUDA API interception library for enforcing per-container VRAM and compute limits
CUDA MPS	NVIDIA Multi-Process Service — enables multiple CUDA contexts to share a GPU with SM partitioning
vGPU	NVIDIA Virtual GPU — hardware GPU slicing available only on enterprise/professional GPUs

MIG	Multi-Instance GPU — hardware GPU partitioning available only on A100/H100/B200
D3cold	Deep PCIe power state that causes the RTX 5090 reset bug in VM passthrough scenarios
WebRTC	Web Real-Time Communication — browser-native protocol for low-latency video/audio streaming
NVENC	NVIDIA hardware video encoder used for desktop stream encoding
ZFS	Advanced filesystem with snapshots, checksums, RAIDZ, and compression
cgroups v2	Linux kernel feature for resource limitation and isolation (CPU, memory, I/O)